

SLIDE 1 OF 33 · STARTS 0:00 · 41 WORDS

- Good morning, and thanks for spending this slot with me.
- Today is about docs that notice when they've broken and help fix themselves.
- It's also about the instructions we hand our AI agents, which turn out to need the same treatment.



YOUR SPEAKER

Manny Silva

Head of AI Docs Practice, Promptless

15 YEARS	Building documentation for Apple, Google, and Skyflow
OPEN SOURCE	Created Doc Detective, a documentation testing tool
RESEARCH	Co-authored DoGBench, the Documentation Generation Benchmark
BOOKS	*Docs as Tests*, *Docs as Tests & AI*, and the forthcoming *Docs as Tests & Hardware*

linkedin.com/in/manuelrbsilva docsastests.com instructionmanuel.com

SLIDE 2 OF 33 · STARTS 0:18 · 134 WORDS

- A little about me.
- I'm Manny Silva, and I've spent about fifteen years building documentation at Apple, Google, and Skyflow.
- These days I lead the AI docs practice at Promptless.
- I codified the Docs as Tests strategy, which treats every doc as a set of claims you can test against the product.
- I created Doc Detective, an open-source tool for running those tests.
- I've written two books on it, and the second, Docs as Tests and AI, is where most of today's material comes from.
- A third, on hardware, is on the way.
- On the research side, I co-authored DoGBench, which measures how well AI agents write and maintain real user-facing docs.
- I like diving into the deep end as the zeroth user, so most of what I'll show you, I've broken myself first.

Source: speaker bio as submitted to LavaCon (presentations/lavacon-2026/proposal.md) and lavacon.org/blog/speaker/manny-silva.

11:47 PM

A release renames a button. Nobody tells the docs team.

PR #4521 merges. `Login` becomes `Sign In`. Three pages and one screenshot still say `Login`.

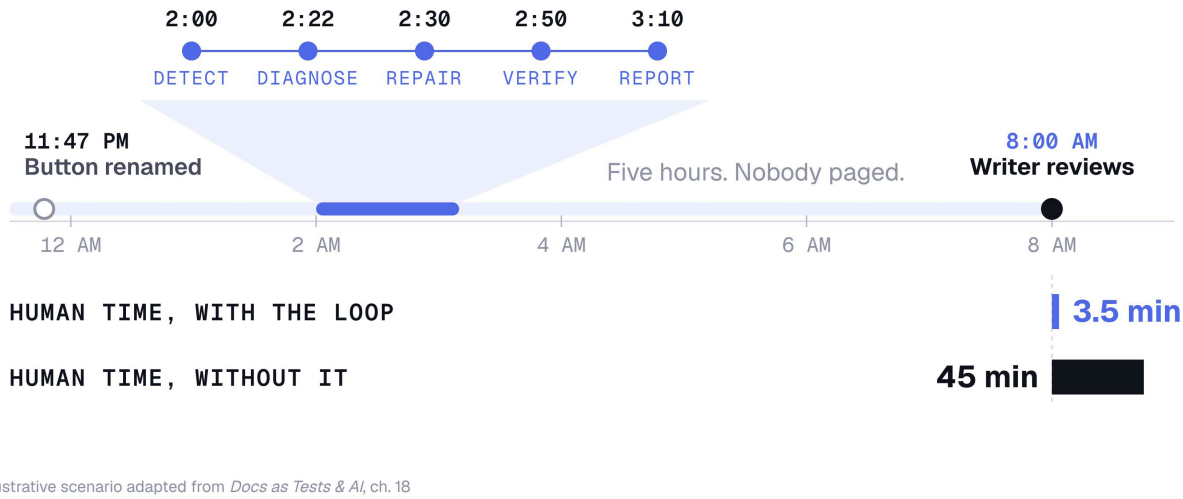
SLIDE 3 OF 33 · STARTS 1:15 · 114 WORDS

- It's 11:47 on a weeknight.
- An engineer merges a UI cleanup, pull request 4521.
- One of the changes renames a button from Login to Sign In.
- It's a good change, and nobody tells the docs team, because nobody thinks of it as a docs change.
- Three pages in your getting-started content still say Login, and one screenshot still shows it.
- Most of you have lived the next morning.
- You find out from a support ticket, or a customer, or not at all.
- This scenario comes from my book, but I'd bet everyone here has a real version of it.
- So let's follow that button through a night where the docs take care of themselves.

Source: Docs as Tests & AI, ch. 18 (illustrative scenario).

THE NIGHT SHIFT

By 8 AM, the fix is waiting for review

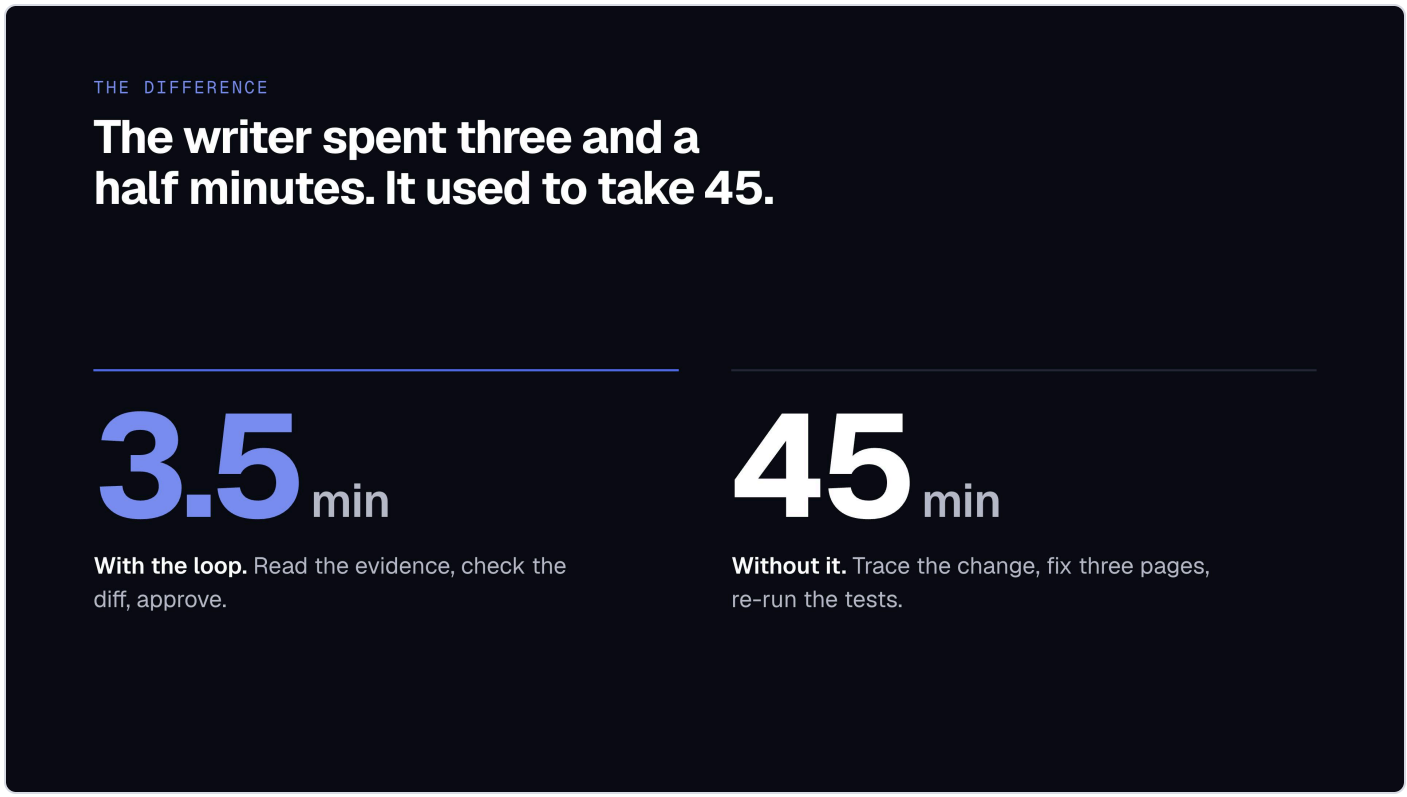


Illustrative scenario adapted from *Docs as Tests & AI*, ch. 18

SLIDE 4 OF 33 · STARTS 2:04 · 174 WORDS

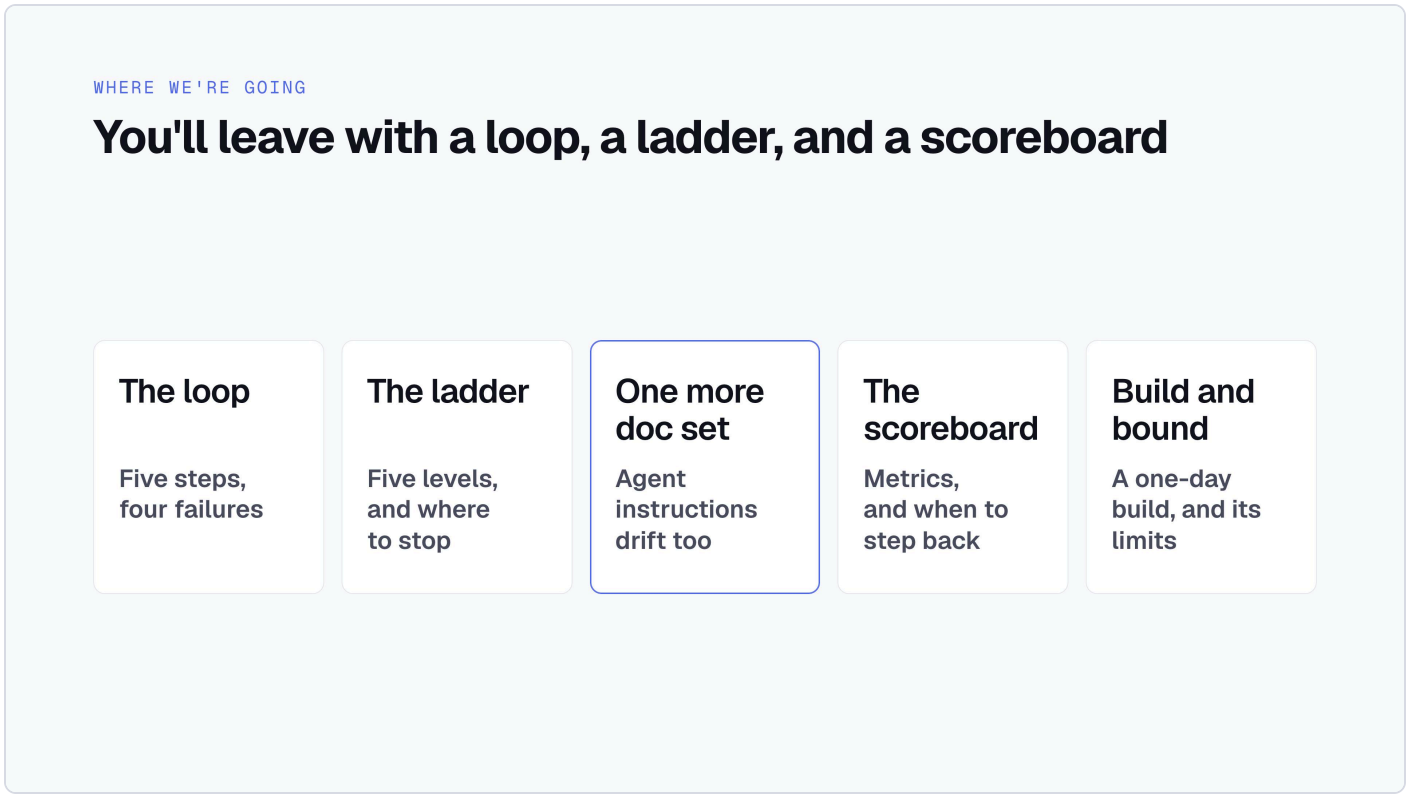
- At two in the morning, the nightly doc tests run.
- The test that clicks Login can't find it, and a screenshot comparison fails.
- That's detect.
- By 2:22, an agent has traced the failure to pull request 4521 and found every page that mentions the old label.
- At 2:30 it opens a proposed change that updates all three pages and swaps in the new screenshot.
- In Git that's a pull request, and in a CCMS it's a draft in someone's review queue.
- At 2:50 it runs the tests again against its own fix, and everything passes.
- At 3:10 it posts the summary and the evidence to the docs channel.
- Then nothing happens for five hours, and nobody gets paged.
- At eight, the writer opens the change, reads the evidence, and approves it in about three and a half minutes.
- The bottom row is the same morning without the loop.
- You spend twenty minutes finding the cause, fifteen fixing three pages, and ten re-running tests.
- It's 8:45 before you start the work you planned for today.

Source: Docs as Tests & AI, ch. 18 timeline (detect through report times as published). The book's review time is 9:30 AM; this talk moves review to 8:00 AM to match the session title.



SLIDE 5 OF 33 · STARTS 3:18 · 73 WORDS

- Three and a half minutes of human time with the loop, and forty-five without it.
- The writer still did real work.
- Their job was reviewing, and "does this fix look right?" is a quicker question than "what broke, and how do I fix it?"
- That's about forty minutes back for every break.
- Count the breaks your docs took in your last release, and you'll have a rough sense of what this is worth.



SLIDE 6 OF 33 · STARTS 3:50 · 107 WORDS

- Here's the plan for the next half hour.
- First, the loop itself, and how it adapts to four failures you already have.
- Second, a maturity ladder, so you can place your team on it and decide how far to climb.
- Third, one more set of documentation that most teams don't count yet.
- Fourth, the metrics that say a system is working, and the ones that say it's time to take autonomy away.
- And last, a version you can build in about a day.
- One note for the structured-authoring folks.
- These tests run against your published output, so it doesn't matter whether your source is Markdown or DITA.

THE FIVE STEPS

Each step earns its place by what it hands to the next



SLIDE 7 OF 33 · STARTS 4:36 · 140 WORDS

- First, a quick definition, because "self-healing" gets used loosely.
- A self-healing system responds to damage it detected and leaves working pages alone.
- For docs, the damage is drift, the gap between what a page says and what the product does.
- The loop has five steps, and each one hands something concrete to the next.
- Detect hands over a failure that names the test, the assertion, and the page.
- Diagnose hands over a root cause, a scope, and a confidence level.
- Repair hands over a narrow change, with every edit linked to evidence.
- Verify proves the fix works and didn't break anything else.
- Report packages it all for a person, in the right channel.
- If a step doesn't produce something the next step can use, it isn't doing work.
- Your tests power detect and verify, and agents power diagnose and repair.

Source: Docs as Tests & AI, ch. 17 (five-step loop) and ch. 18 ("what each step produces").

DETECT

Diagnosis is only as good as the test description

THE AGENT GETS

```
Test 47  FAILED
```

Starts from nothing

THE AGENT NEEDS

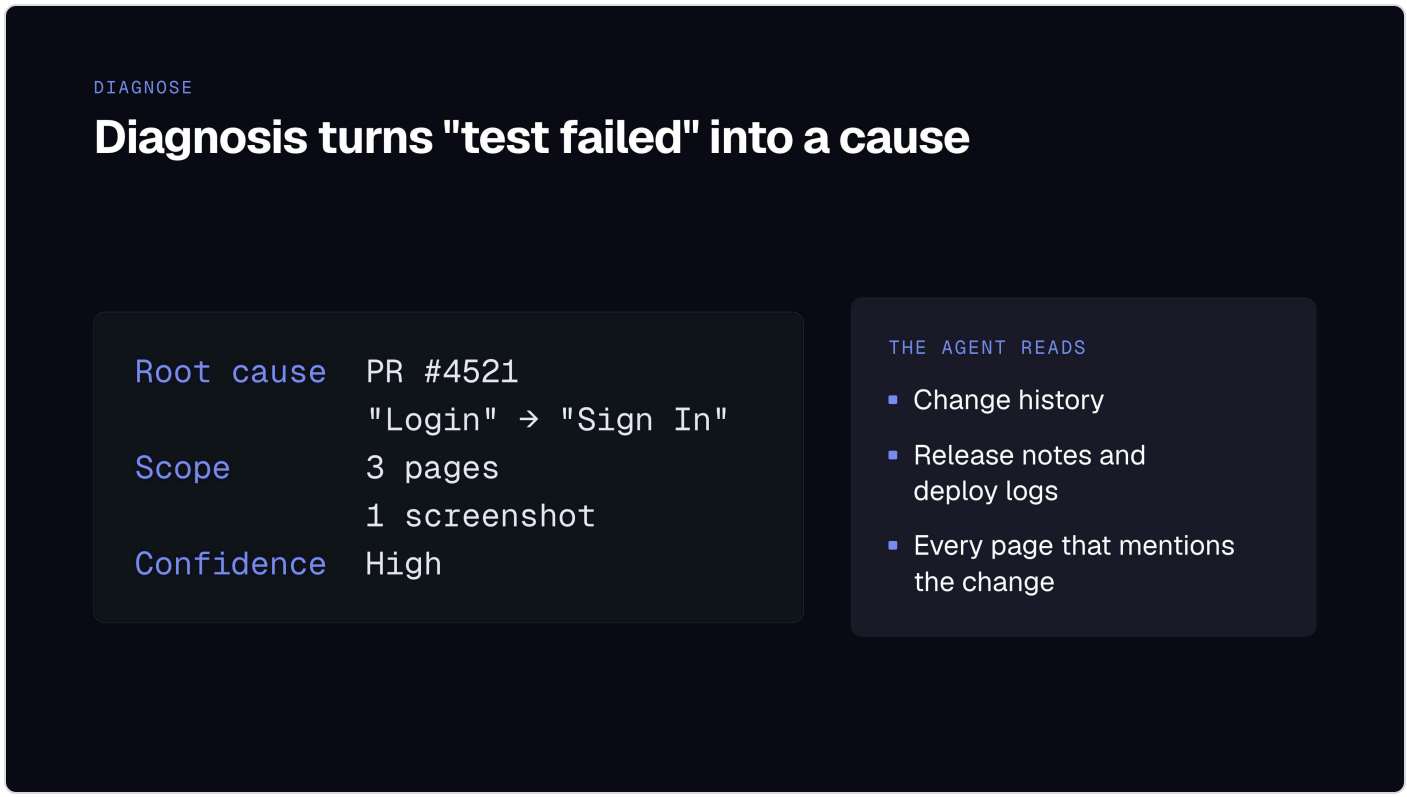
```
Login button is present
on getting-started page
FAILED
"Login" not found
```

Half the diagnosis is done

SLIDE 8 OF 33 · STARTS 5:36 · 100 WORDS

- Detection quality sets the ceiling for everything after it.
- The agent doing diagnosis at two in the morning only knows what your test told it.
- If your test is called Test 47, the agent starts from nothing.
- If it says "login button is present on the getting-started page," and the failure says Login wasn't found, half the diagnosis is done.
- So name your tests for the agent that'll read them.
- That's a writing problem, and writers already know how to solve it.
- If a new teammate couldn't act on the failure without asking you a question, an agent can't either.

Source: Docs as Tests & AI, ch. 18 ("Verify login button is present on getting-started page" vs. "Test 47").



SLIDE 9 OF 33 · STARTS 6:18 · 121 WORDS

- Diagnosis turns "a test failed" into a cause.
- The agent looks at what changed recently, like merged pull requests, deploy logs, and release notes.
- It ties the failure to one change, PR 4521.
- Then it searches the docs for every page that mentions the old label, including pages whose tests didn't fail.
- It also reports a confidence level, and here it's high, because a deterministic test failed and maps straight to a code change.
- That confidence decides how much scrutiny the fix gets later.
- Two practical notes.
- Retry a flaky UI failure before anyone diagnoses it.
- And the agent needs read access to your product's change history, which is usually a conversation with engineering and often the slowest part of setup.

Source: Docs as Tests & AI, ch. 17 (Diagnose; scanning beyond testable claims) and ch. 18 (root cause, scope, confidence).

REPAIR AND VERIFY

Repair narrowly, then prove it before anyone looks

REPAIR

```
- Login
+ Sign In
```

3 pages, 1 screenshot, then stop

VERIFY

- ✓ **Failing test passes**
- ✓ **Full suite passes**
- ✓ **Screenshot matches**

SLIDE 10 OF 33 · STARTS 7:10 · 145 WORDS

- Repair is all about scope.
- Swap Login for Sign In on three pages, replace one screenshot, and stop.
- Every change links back to the diagnosis, and if the agent couldn't fix something, it says so.
- Then verify re-runs the test that failed, and then the whole suite.
- A fix to one page can break an assertion on another, and you want to catch that at three in the morning, before a person spends any attention on it.
- If you work in structured content, there's good news here.
- If the label lives in a variable or a key, the fix is one edit.
- Teams that run these tests notice a side effect quickly.
- Fabian Rodriguez at Kong put it this way: "They made a change to the API, they made a change to this other thing. We were able to catch those because of the tests."

Source: Docs as Tests & AI, ch. 18 (Repair, Verify, regression check). Rodriguez quote as published at https://www.docsastests.com/kong-case-study (the book's ch. 18 version splices two separate quotes; this is the single original quote).

REPORT

A self-healing pull request carries its own evidence

```
### Self-Healing Fix: UI string update

Root cause:    PR #4521 (UI cleanup) renamed
               button "Login" → "Sign In"
Trigger:       Nightly test failure: button
               with text "Login" not found
Changes:       3 pages
  getting-started.md   2 instances
  authentication.md    1 instance
  troubleshooting.md   caption, screenshot
Verification: 47 tests passing, 0 regressions
Limitations:   None. String swap only.
Attribution:   Generated by self-healing system
```

Summary	One line: what changed and why
Root cause	The product change that triggered it
Evidence	Failing run, product PR, correlation
Changes	The diff, scoped narrowly
Verification	What passed, what warned
Limitations	What it couldn't fix, and why
Attribution	A clear marker that a system wrote it

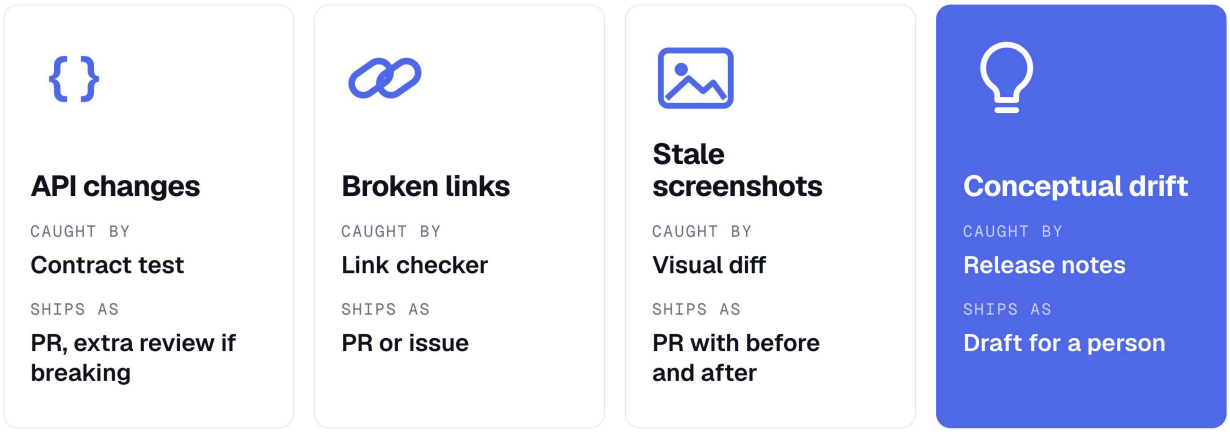
SLIDE 11 OF 33 · STARTS 8:12 · 88 WORDS

- At eight, this is what the writer sees.
- A colleague can explain their reasoning in a review thread, but the system can't, so the pull request has to explain itself.
- It has seven parts: a summary, the root cause, the evidence, the changes, the verification results, the limitations, and an attribution line.
- Limitations and attribution are the two I see teams skip.
- Limitations keep a reviewer from assuming a fix is complete when it isn't.
- Attribution reminds them they're checking the agent's judgment as much as the prose.

Source: Docs as Tests & AI, ch. 18 ("Effective self-healing PRs include" and the example PR description).

FOUR FAILURES YOU ACTUALLY HAVE

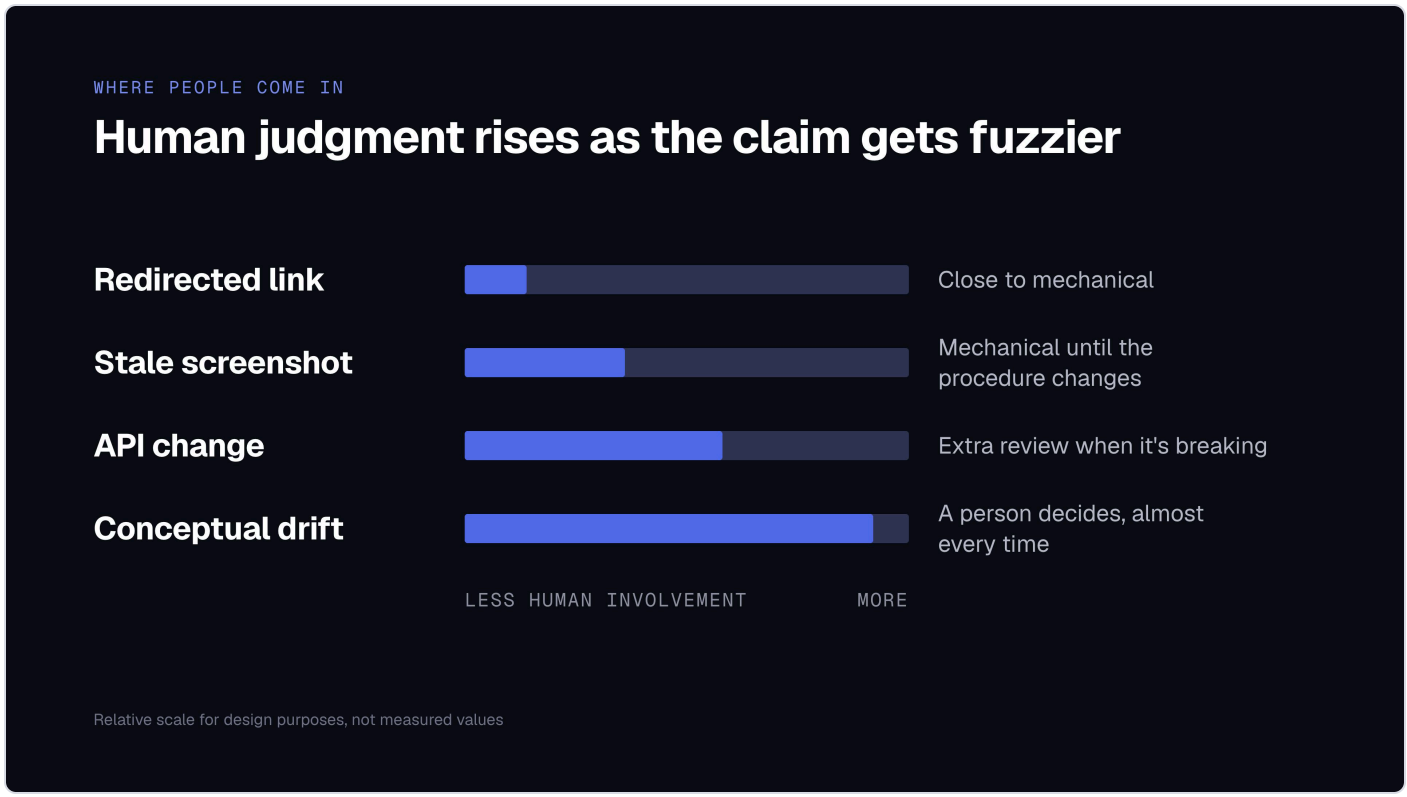
Same loop, different middle



SLIDE 12 OF 33 · STARTS 8:50 · 131 WORDS

- A renamed button is the easy case, on purpose.
- Here's the same loop applied to four failures you actually have.
- The steps stay the same, but the middle changes, and so does how much a person gets involved.
- For API changes, a contract test or a spec diff catches it, and breaking changes get extra review.
- For broken links, the interesting part is diagnosis.
- If the target redirected, swap the URL.
- If it was deleted, a person decides, because the sentence around it might need to change.
- For screenshots, the agent recaptures the image if only the visuals changed, and flags it if the procedure changed.
- Conceptual drift is the hard one.
- Tests rarely catch it, so detection comes from release notes and feature flags, and a person makes the call.

Source: Docs as Tests & AI, ch. 18 ("Adapting the Flow to Other Scenarios").

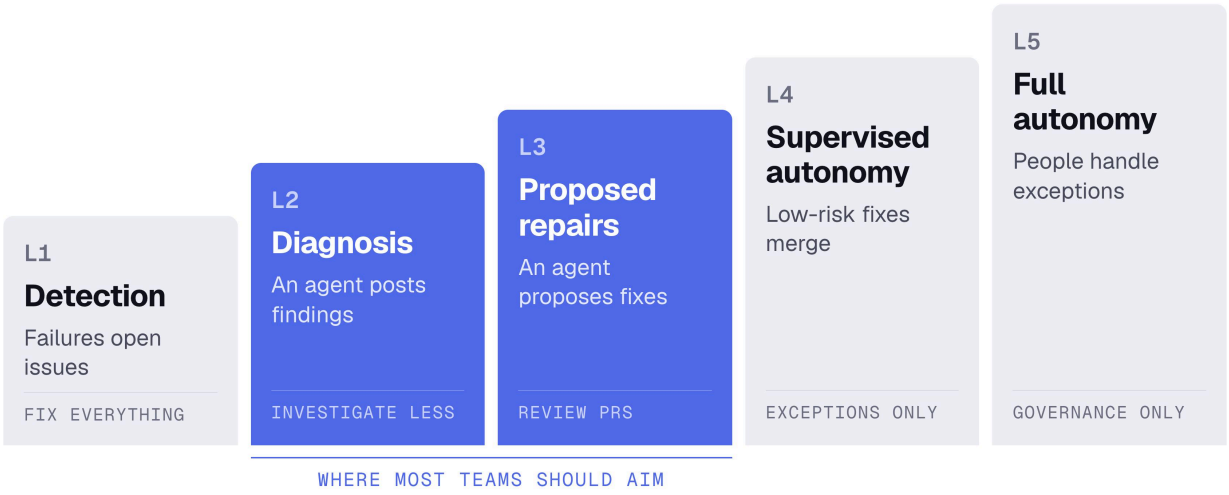


SLIDE 13 OF 33 · STARTS 9:46 · 62 WORDS

- Across those four, human judgment goes up as the claim gets fuzzier.
- A redirected link is close to mechanical.
- Conceptual drift needs a person almost every time.
- The bars are relative, and they show the shape you should design for.
- Your conceptual docs shouldn't get the same autonomy as your link checker, and deciding that is what the maturity ladder is for.

THE MATURITY LADDER

Five levels between "tests file a ticket" and "it merges its own fixes"



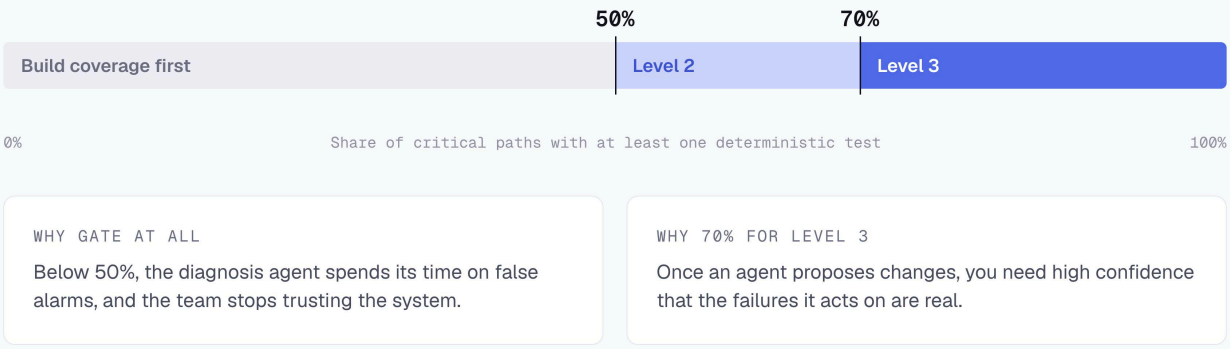
SLIDE 14 OF 33 · STARTS 10:13 · 114 WORDS

- I break self-healing into five levels, and each one builds on the last.
- Level one is detection.
- Failures become issues, and people still fix everything.
- Most teams don't have this yet.
- Level two is diagnosis, where the issue says what changed and which pages it touched.
- Level three is proposed repairs.
- An agent proposes the change, people approve it, and your team's job becomes review.
- At level four, low-risk fixes merge on their own, some after a delay.
- I'd consider level five, full autonomy, only for low-stakes internal docs.
- Levels two and three are where I'd point most teams in this room.
- Take a second and place your own team.
- Where are you today?

Source: Docs as Tests & AI, ch. 17 (maturity model table and level descriptions; "Most teams benefit from targeting Level 2 or Level 3").

BEFORE YOU CLIMB

Deterministic test coverage decides how high you can climb



WHY GATE AT ALL

Below 50%, the diagnosis agent spends its time on false alarms, and the team stops trusting the system.

WHY 70% FOR LEVEL 3

Once an agent proposes changes, you need high confidence that the failures it acts on are real.

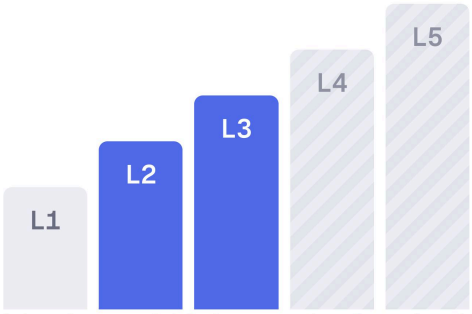
SLIDE 15 OF 33 · STARTS 11:02 · 94 WORDS

- So how do you know you're ready to climb?
- Deterministic coverage.
- A deterministic test gives the same result for the same input, with no AI judging.
- Coverage is the share of your critical paths that have at least one of those tests.
- Below fifty percent, build coverage first, or a diagnosis agent will spend its time chasing false alarms.
- Between fifty and seventy, you're ready for level two, and above seventy, level three.
- You can work out your number on a napkin.
- Count your critical paths, count the ones with deterministic tests, and divide.

Source: Docs as Tests & AI, ch. 17 ("Deterministic Test Coverage": below 50%, 50–70%, above 70%).

WHERE TO STOP

Most teams should target Level 2 or 3



L4 and up need risk tiers, months of proof, and sign-off

SKIP SELF-HEALING WHEN

- **Content is regulated**
- **Coverage is under 50%**
- **Nobody owns the page**
- **One author owns a narrow set**

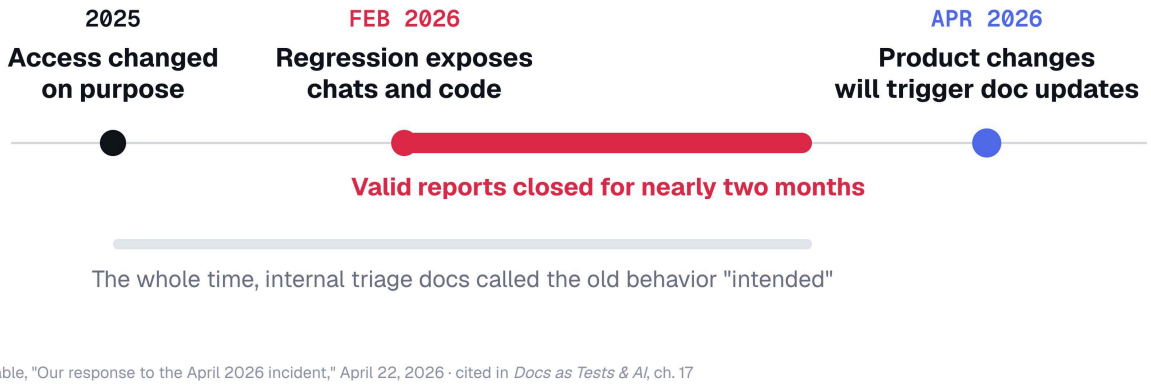
SLIDE 16 OF 33 · STARTS 11:42 · 81 WORDS

- Most teams should target level two or three and stay there a good while.
- Levels four and five need written risk tiers, months of proven reliability, and organizational sign-off.
- There are also places self-healing doesn't belong.
- In regulated content, a person signs off every change, so self-healing can detect and diagnose, but auto-merge is off the table.
- Thin coverage, pages nobody owns, and narrow single-author sets are the others.
- Self-healing pays off when reviewing is cheaper than writing the fix yourself.

Source: Docs as Tests & AI, ch. 17 (Levels 4–5 requirements) and ch. 19 ("When NOT to Pursue Self-Healing").

BEYOND THE DOCS CUSTOMERS READ

The documentation that breaks isn't always customer-facing

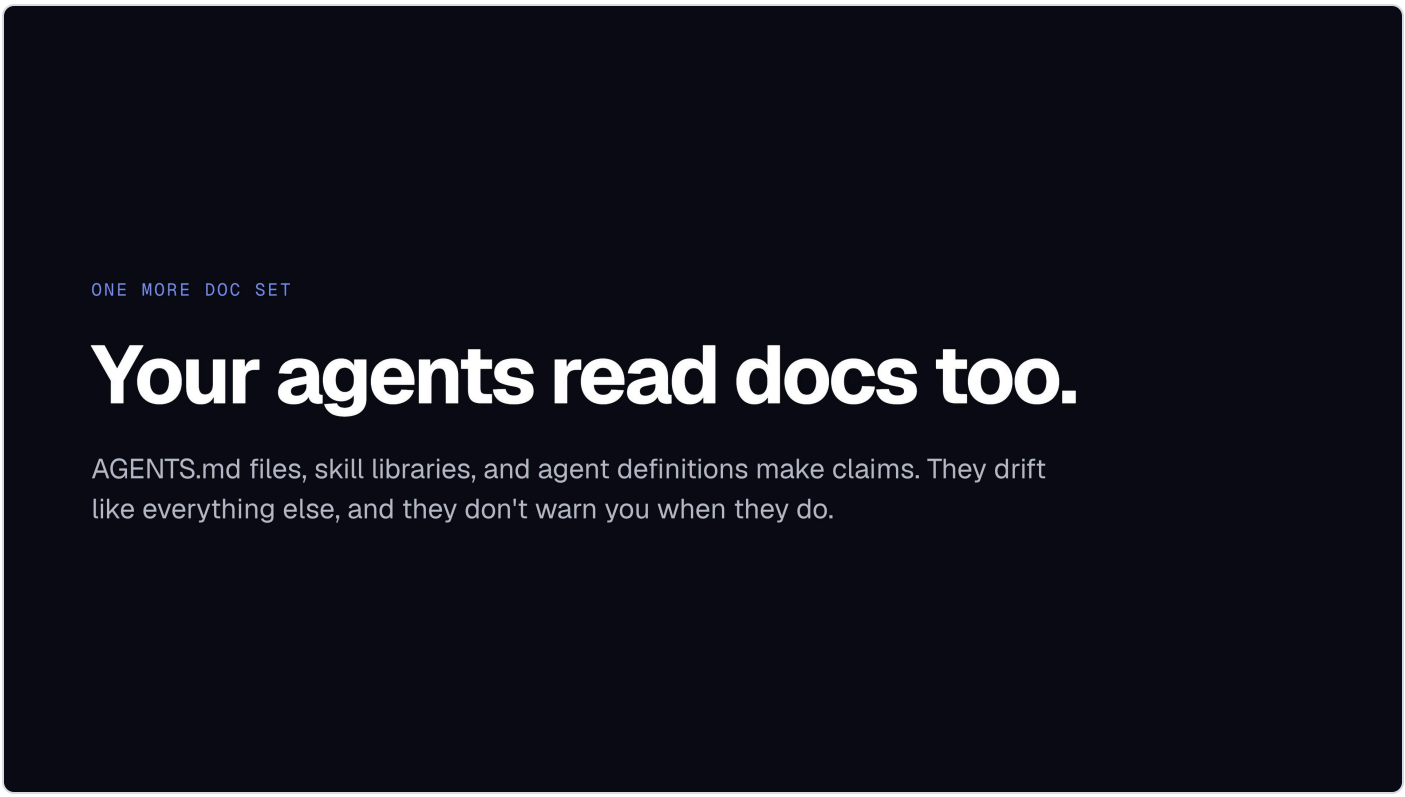


Lovable, "Our response to the April 2026 incident," April 22, 2026 · cited in *Docs as Tests & AI*, ch. 17

SLIDE 17 OF 33 · STARTS 12:17 · 93 WORDS

- Before we go further, here's one real incident, because it changes what counts as documentation.
- In 2025, Lovable deliberately changed who could see project chats.
- Their internal triage docs still described the old behavior as intended.
- Then a regression re-exposed chat histories and source code.
- Researchers reported it, and those reports were closed without escalation for nearly two months.
- The fix they're building is a workflow where product changes to data access trigger an update to the triage docs.
- Those docs had no customers.
- Their readers were people deciding which reports to escalate.

Source: Lovable, "Our response to the April 2026 incident," April 22, 2026. https://lovable.dev/blog/our-response-to-the-april-2026-incident (cited in Docs as Tests & AI, ch. 17).

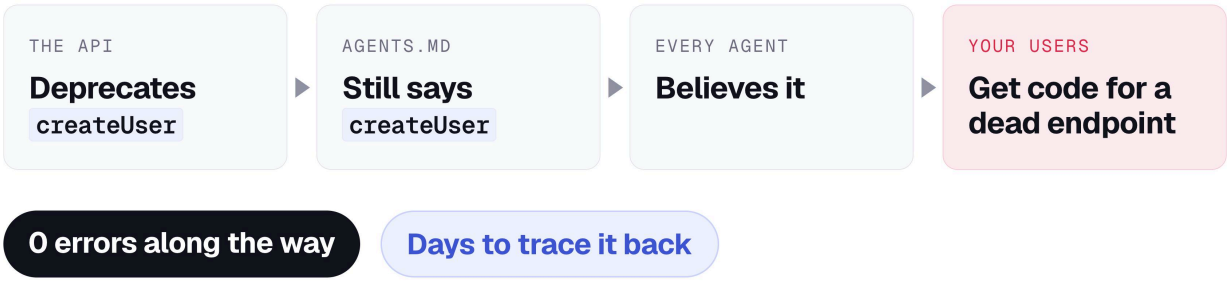


SLIDE 18 OF 33 · STARTS 12:57 · 53 WORDS

- The next doc set has readers that aren't people at all.
- If your team runs AI agents anywhere, there's a file somewhere telling them how to behave.
- That might be an AGENTS.md, a skill library, or agent definitions.
- Those are documents.
- They make claims, and they drift like everything else we've talked about.

CONTEXT ROT

When agent instructions go stale, nothing errors



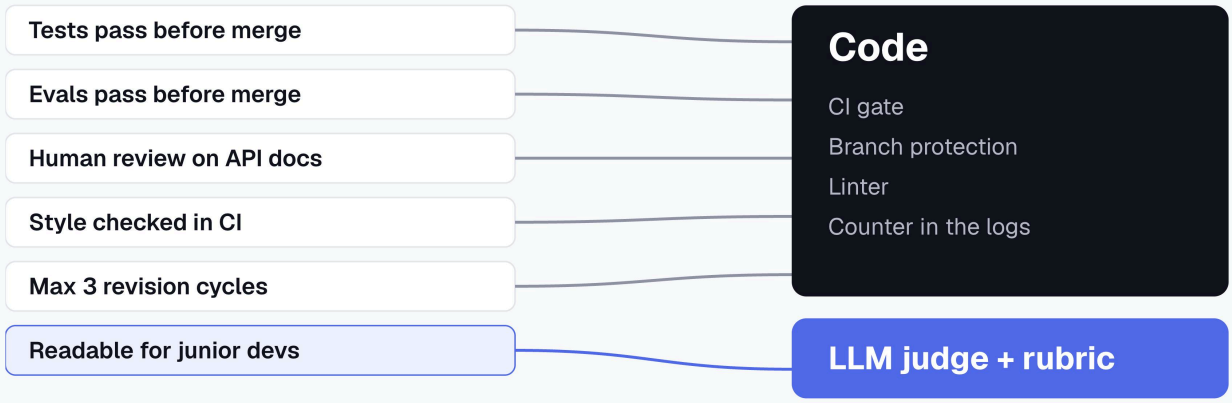
SLIDE 19 OF 33 · STARTS 13:19 · 84 WORDS

- Say your API deprecated an endpoint three weeks ago, and the migration guide shipped.
- Your AGENTS.md still points to the old endpoint.
- Every coding agent that reads it generates code against a deprecated endpoint.
- Nobody notices, because nothing errors.
- The agent did exactly what it was told.
- That's context rot, a confident wrong answer that can take days to trace back to one stale line.
- A person reading an outdated page might notice it feels off, but an agent takes it at face value.

Source: Docs as Tests & AI, Part 3 (ch. 11, 12, 14). Example adapted from a 2026-06-18 article on agent context files.

PROJECT DESCRIPTIONS

Every rule in a project description is already a test



SLIDE 20 OF 33 · STARTS 13:55 · 118 WORDS

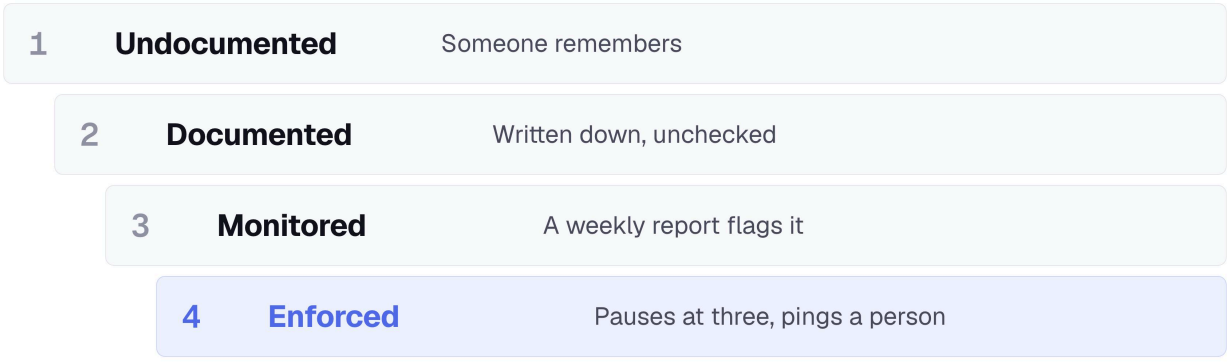
- The good news is that you already know how to test these.
- Every rule in a project description is already a test.
- "All tests pass before merge" is a gate your CI enforces.
- "Human review on API docs" is branch protection plus a CODEOWNERS file.
- A style rule is a linter, and "max three revision cycles" is a counter in your logs.
- Judgment rules, like "readable for junior developers," get an LLM judge with a rubric.
- Pick the cheapest grader that can actually tell pass from fail, which is usually code.
- Matt Nigh at GitHub looked at more than twenty-five hundred agents.md files.
- The ones that worked were specific, and most of the failures were simply too vague.

Source: Docs as Tests & AI, ch. 11 ("How to Test Project Descriptions" and the assertion extraction table). Matt Nigh, "How to write a great agents.md: Lessons from over 2,500 repositories," GitHub Blog, Nov 2025 (the book's ch. 11 summary of this study needs correcting; this wording follows the post). https://github.blog/ai-and-ml/github-copilot/how-to-write-a-great-agents-md-lessons-from-over-2500-repositories/

FROM DOCUMENTED TO ENFORCED

Each instruction rule climbs its own ladder

Example rule: "Maximum 3 AI revision cycles before human takeover"

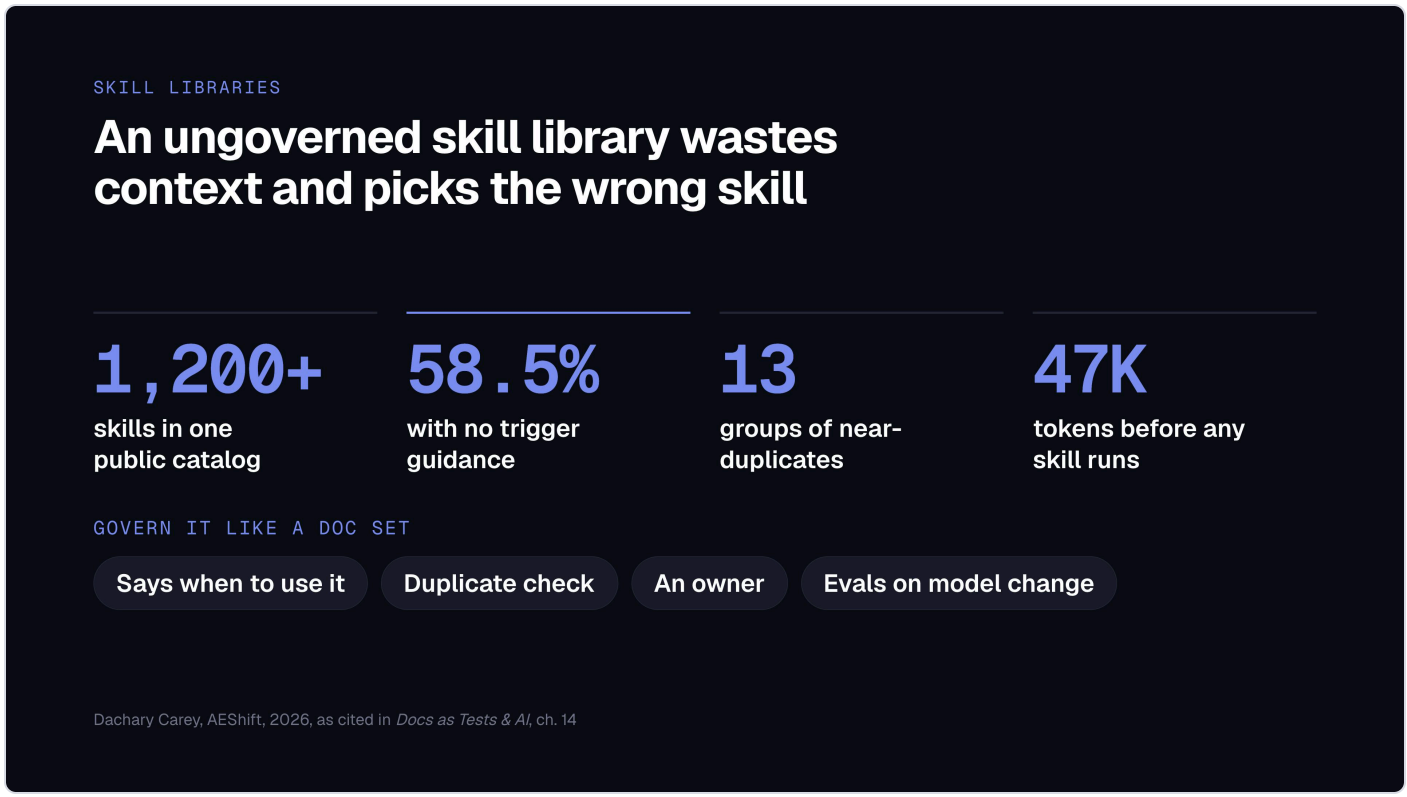


Only stage 4 catches it when nobody is watching.

SLIDE 21 OF 33 · STARTS 14:46 · 75 WORDS

- Instruction rules climb a ladder too.
- Take that three-revision-cycle rule.
- At first it's undocumented, and people follow it because someone remembers last quarter's runaway loop.
- Then it's written in AGENTS.md, and nothing checks it.
- Then a weekly report flags when a workflow goes past three cycles.
- Finally the orchestrator counts, pauses at three, and pings a person.
- Only that last stage catches it when nobody happens to be watching.
- Move one rule at a time.

Source: Docs as Tests & AI, ch. 11 ("From Documentation to Enforcement").



SLIDE 22 OF 33 · STARTS 15:18 · 125 WORDS

- Skill libraries have their own failure mode, and it's growth.
- Dachary Carey analyzed a public catalog of more than twelve hundred skills.
- Fifty-eight and a half percent had no trigger guidance at all, and thirteen groups were near-duplicates.
- The catalog alone cost forty-seven thousand tokens before a single skill ran.
- Five well-chosen skills cost about a hundred and twenty-five.
- If you've ever inherited four pages covering the same procedure, this will feel familiar.
- Govern it like a doc set.
- Every skill says when to use it, new skills get a duplicate check, each one has an owner, and evals re-run on every model change.
- That last part matters because models absorb techniques, so a skill that was essential six months ago can quietly become redundant.

Source: Dachary Carey, "Skills aren't a cheat code for coding agents. They're configuration drift waiting to happen," AEShift, 2026. https://aeshift.com/posts/2026-03-17-swe-skills-bench-do-agent-skills-actually-help-in-real-world-software-engineerin/ (cited in Docs as Tests & AI, ch. 14).

AGENT DEFINITIONS

Tool access is a claim you can test

OVERPRIVILEGED EDITOR

`tools: Read, Write, Grep, Glob, Bash, Browser`

Fixes what it should only flag

SCOPED EDITOR

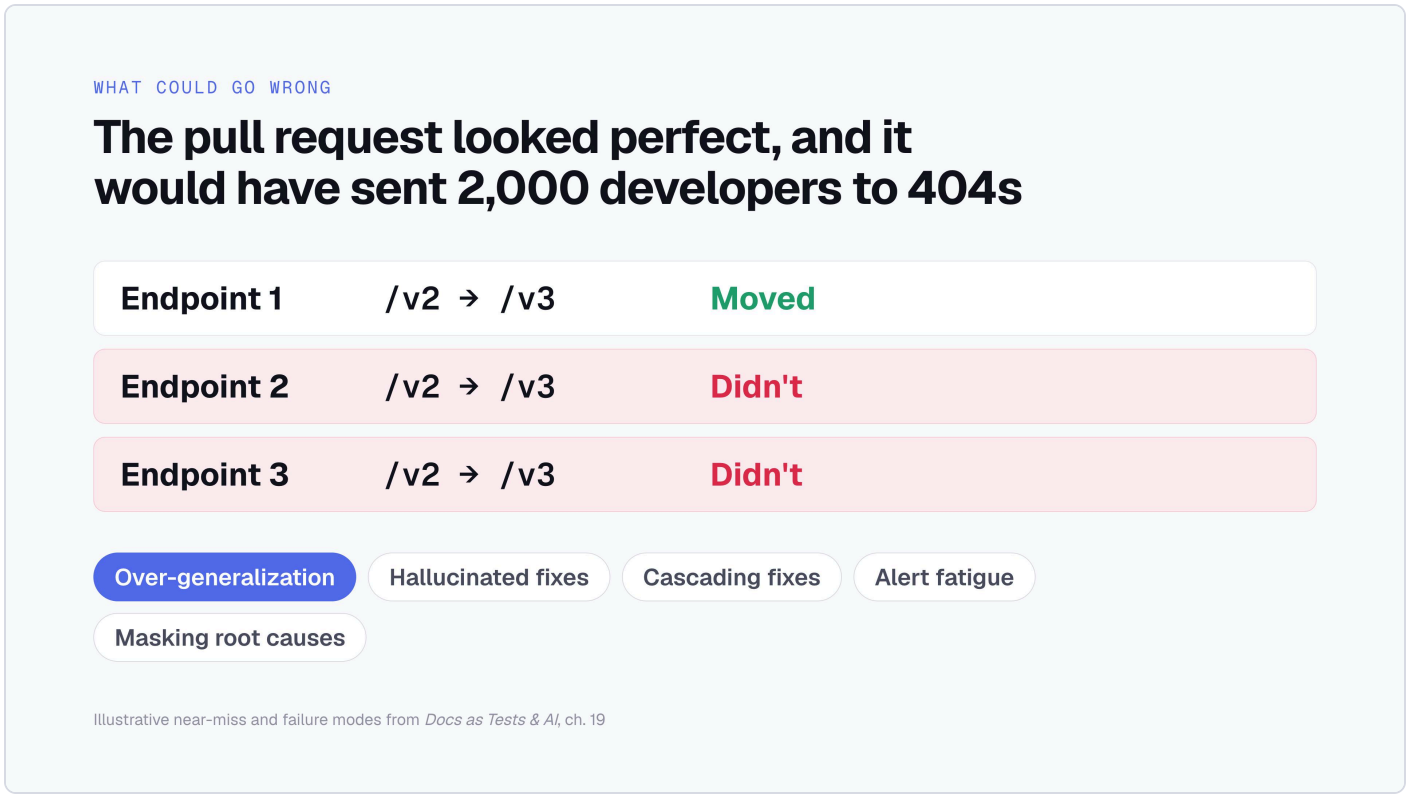
`tools: Read, Grep, Glob`

Produces review comments

SLIDE 23 OF 33 · STARTS 16:12 · 81 WORDS

- Agent definitions make claims about capability, and those are testable too.
- Give an editing agent write access, and sooner or later it'll fix a code sample it was only supposed to review.
- Read access produces review comments, and write access produces edits.
- When one agent hands work to another, the second should get a subset of the first one's tools.
- Checking all of this is deterministic.
- Compare the tools each role declares against what the role needs, and flag the gaps.

Source: Docs as Tests & AI, ch. 12 ("The Tools Field as a Capability Boundary" and "Tool Access Is Testable").



SLIDE 24 OF 33 · STARTS 16:46 · 110 WORDS

- Before metrics, here's why you need them.
- A writer wakes up to a self-healing pull request, green checkmarks everywhere, and almost approves it from bed.
- Then she notices it moved three endpoints from v2 to v3, and the migration only moved one.
- The tests passed, because they only covered the endpoint that actually moved.
- If she'd approved it, two thousand developers would have hit errors on endpoints that were fine.
- That's over-generalization, one of the most common ways these systems fail.
- The others are on the slide.
- Every one of them shows up in your numbers before it shows up in a customer complaint, if you're measuring the right things.

Source: Docs as Tests & AI, ch. 19 (opening near-miss; "What Could Go Wrong" table; over-generalization prevention strategies).

THE SCOREBOARD

Four numbers tell you whether the healing system is healthy

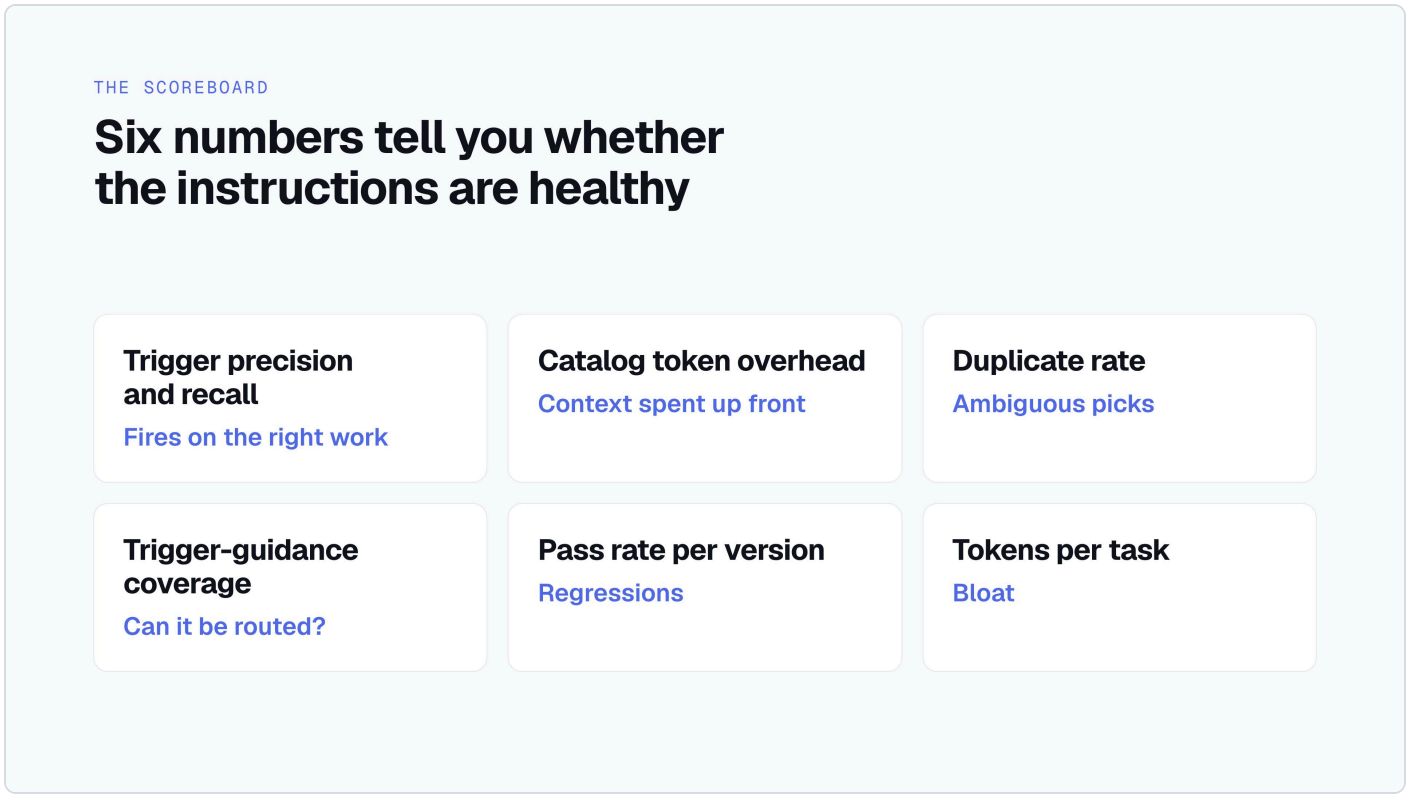
	HEALTHY	WARNING	CRITICAL
Accuracy rate	85%+	80–85%	Under 80%
Rollback rate	Under 3%	3–5%	Over 5%
Over-generalization	Under 5%	5–10%	Over 10%
Fix rate	50%+	30–50%	Under 30%

Starting defaults from *Docs as Tests & AI*, ch. 19. Tighten them for external API reference.

SLIDE 25 OF 33 · STARTS 17:33 · 97 WORDS

- So, four numbers for the healing system.
- Accuracy is the share of pull requests approved without edits.
- Watch it under eighty-five percent, and act under eighty.
- Rollback is merged changes that later get reverted, with a warning above three percent and critical above five.
- Over-generalization is how often a reviewer has to narrow the scope, with a warning above five and critical above ten.
- Fix rate is how many detected issues get resolved, with a warning below fifty and critical below thirty.
- These are defaults, so tune them, write them down, and borrow your engineering team's tooling.

Source: Docs as Tests & AI, ch. 19 ("Monitoring the Healing System" table).



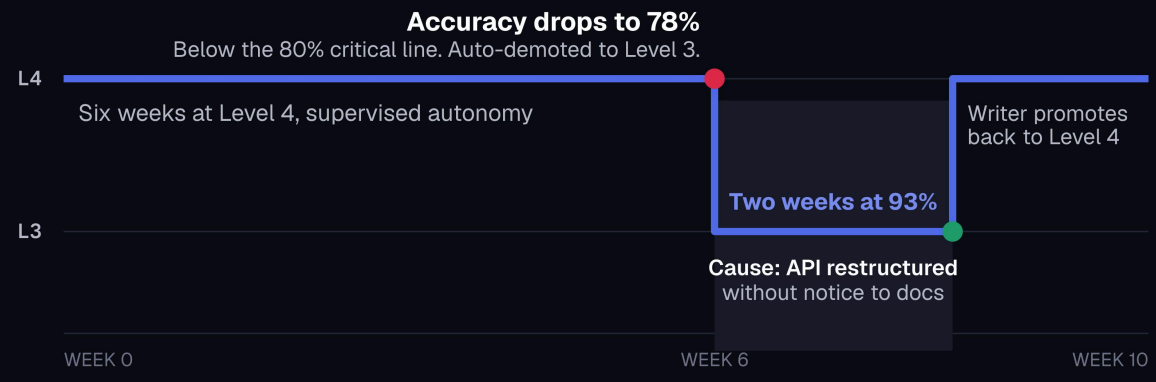
SLIDE 26 OF 33 · STARTS 18:15 · 88 WORDS

- The instructions need their own scoreboard.
- Trigger precision and recall tell you whether a skill fires on the right work, and only that.
- Token overhead is what you pay before any instruction does work, and duplicates make the agent's choice ambiguous.
- An instruction with no trigger guidance can't be routed or tested.
- And when you track pass rate and tokens per version, a drop is a regression and a climb is bloat.
- On my own team, every skill change goes through review, the same as a docs change.

Source: Docs as Tests & AI, ch. 14 (eval dimensions, triggering accuracy, pass rate and token tracking across versions) and Carey (catalog overhead, duplicates, trigger guidance).

A DEMOTION

Demotion is how the system tells you something changed upstream



Payments API reference, illustrative scenario from *Docs as Tests & AI*, ch. 19

SLIDE 27 OF 33 · STARTS 18:53 · 106 WORDS

- Here's what happens when one of those numbers slips, in a scenario from the book.
- The payments API reference has been at level four for six weeks.
- One morning, accuracy drops to seventy-eight percent, under the critical line.
- The system demotes payments to level three on its own and alerts the writer.
- The investigation finds the real cause, an API restructuring nobody told the docs team about.
- The docs were the symptom.
- After the fix, the system holds ninety-three percent for two weeks, and the writer promotes it back.
- Promotion runs on evidence too, like accuracy above ninety percent for a month and rollbacks under three.

Source: Docs as Tests & AI, ch. 19 (payments demotion example, promotion criteria, vetting period). The chart's week positions after week 6 are approximate; the book gives "six weeks" and "two weeks at 93%".

WHAT THE STORY TEACHES

Autonomy moves in both directions, one content area at a time

L4 → L3

Demotion keeps autonomy honest

Automatic demotion makes promotion safe

L4 · L3 · L2

Trust is per content area

Each area earns its own level

3 in 30

Repeat healing is a product signal

3 fixes to one page in 30 days? Send a person.

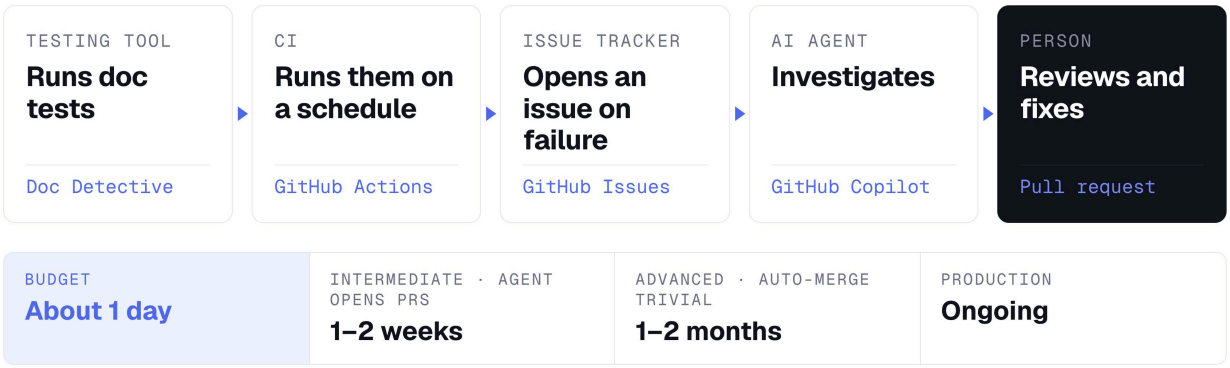
SLIDE 28 OF 33 · STARTS 19:38 · 82 WORDS

- That story teaches three things.
- Demotion keeps autonomy honest, and it's the safety net that makes promotion safe.
- Trust is per content area, so one pipeline can run at level four on the API reference and level two on conceptual docs.
- And repeated healing is a product signal.
- If the same page needs three fixes in thirty days, send a person before the system makes a fourth.
- A self-healing system that hides product bugs is doing harm, however good its accuracy looks.

Source: Docs as Tests & AI, ch. 19 (per-area trust; "Masking root causes": three fixes to the same page in 30 days).

BUILD IT

You can build a basic Level 1 and 2 loop in about a day



BUDGET	INTERMEDIATE · AGENT OPENS PRS	ADVANCED · AUTO-MERGE TRIVIAL	PRODUCTION
About 1 day	1–2 weeks	1–2 months	Ongoing

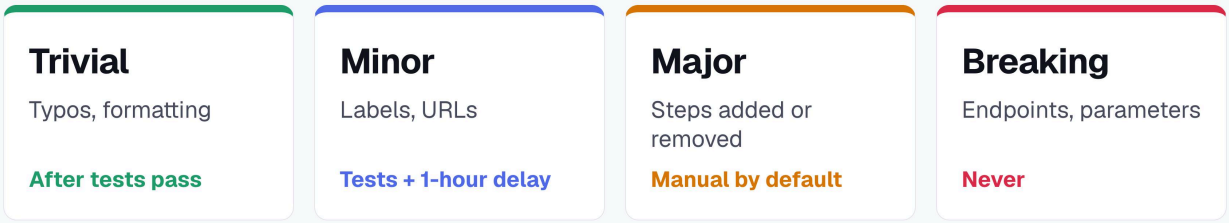
SLIDE 29 OF 33 · STARTS 20:13 · 122 WORDS

- Here's the version you can build this week.
- A testing tool runs your doc tests.
- I built Doc Detective, but a Playwright suite your engineers already maintain works too.
- CI runs the tests on a schedule, and a failure opens an issue from a template.
- An agent gets assigned to the issue and posts what it found, and a person writes the fix.
- That's about a day of setup, and tuning the diagnosis takes a few more weeks.
- Every box is swappable, so use GitLab or Jenkins, Jira or Linear, and whichever agent can see your content and change history.
- To keep costs down, run in the pipeline engineering already pays for, prefer deterministic tests, and skip diagnosis when nothing upstream changed.

Source: Docs as Tests & AI, ch. 18 ("The Low-Effort Entry Point" and "Scaling Beyond Budget") and ch. 19 ("Cost-Conscious Implementation").

BOUND IT

Put merge permissions in writing before you need them

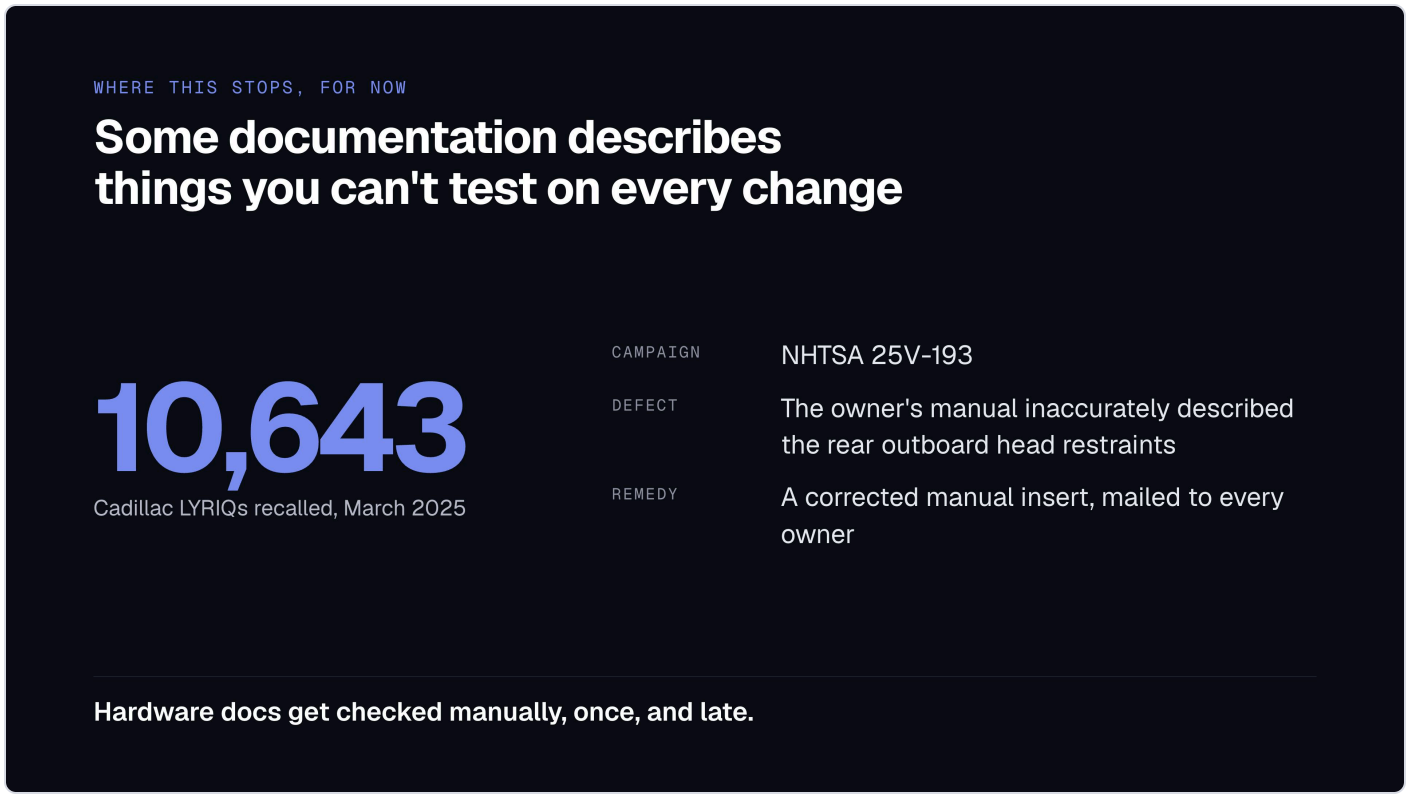


Product names aren't trivial. Regulated content is always manual.

SLIDE 30 OF 33 · STARTS 21:06 · 87 WORDS

- Before you let anything merge on its own, write down what's allowed to.
- Trivial changes, like typos and formatting, can merge after tests pass.
- Minor changes, like labels and URLs, get tests plus an hour's delay, so someone can step in.
- Major changes, like steps added or removed, default to human review.
- Breaking changes, like endpoints and parameters, never auto-merge.
- Two caveats.
- Product names aren't trivial unless you've tested how the agent handles near-miss terms, and regulated content is always manual.
- This week, write your never-auto-merge list.

Source: Docs as Tests & AI, ch. 19 ("Risk Tiers" table, product-name note, "Try This" exercise).



SLIDE 31 OF 33 · STARTS 21:43 · 95 WORDS

- One contrast before I wrap up.
- Everything today assumed text you can test on every change, and a physical product isn't that.
- In March 2025, GM recalled 10,643 Cadillac LYRIQs.
- The federal record, campaign 25V-193, says the owner's manual described the rear head restraints wrong.
- The hardware was fine, and the remedy was a corrected insert mailed to every owner.
- Hardware teams check their docs too, but manually, once, and late.
- Making those checks repeatable is the book I'm writing now.
- If you document hardware and have a story like this, come find me afterward.

Source: NHTSA recall 25V-193 (General Motors, March 2025; 10,643 vehicles; owner's manual head restraint description). As used in the CIDM 2026 deck.

START MONDAY

Four things to do next

Rename your tests for the agent that reads them

What was tested, where, and what it expected.

Wire one failing test to an issue and an agent

A basic Level 2 loop, in about a day.

Treat AGENTS.md and skills as a doc set

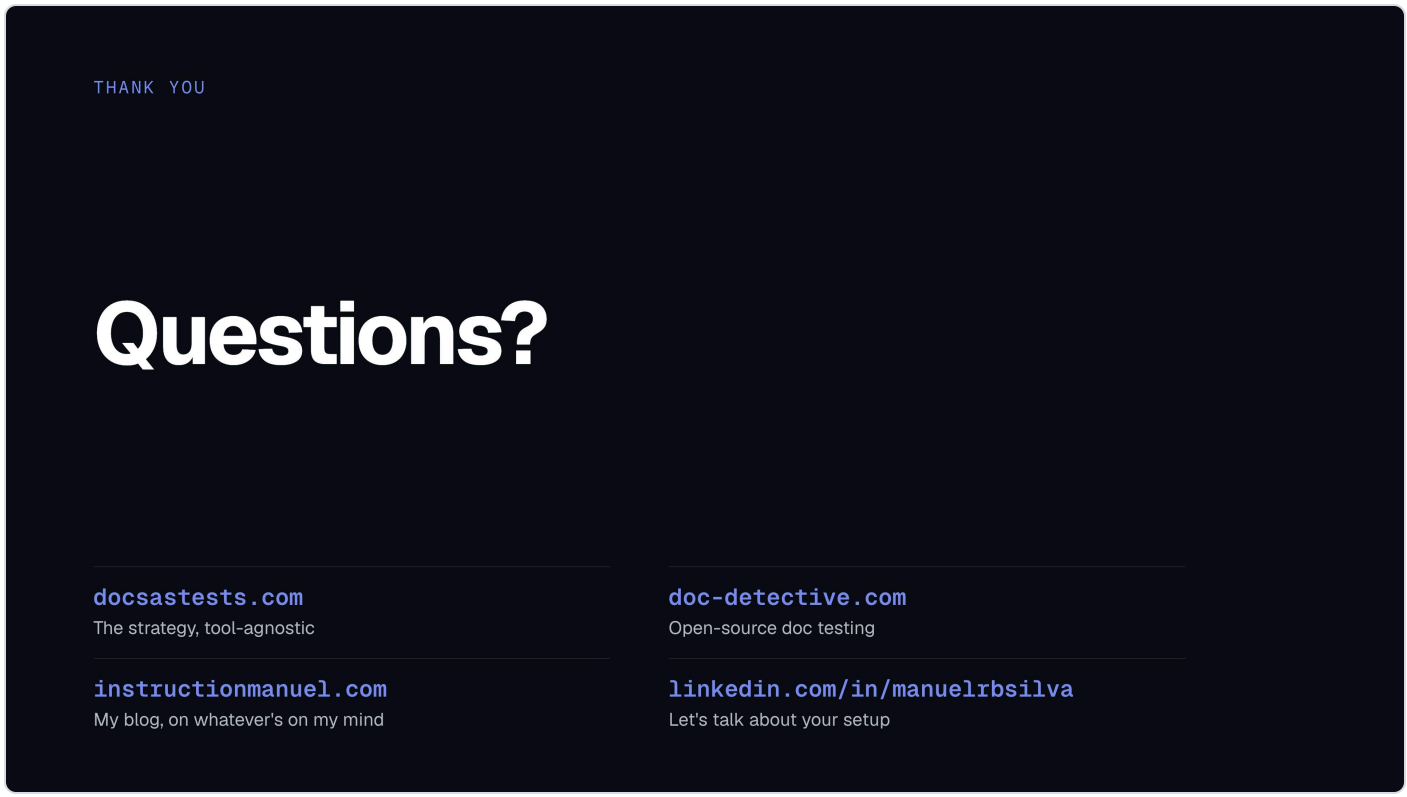
Extract their rules. Test the ones a machine can check.

Write thresholds and a never-auto-merge list

Before you climb, while things are calm.

SLIDE 32 OF 33 · STARTS 22:24 · 70 WORDS

- If you do four things after today, do these.
- Rename your tests for the agent that reads them.
- Wire one failing test to an issue and an agent, which gets you a basic level two in about a day.
- Treat your AGENTS.md and your skills like any other doc set, and test their rules.
- And before you climb, write down your thresholds and your never-auto-merge list, while things are calm.



SLIDE 33 OF 33 · STARTS 22:54 · 51 WORDS

- Thank you. docsastests.com has the strategy material, and doc-detective.com is the open-source tool. instructionmanuel.com is my blog, where I write about whatever's on my mind.
- Docs as Tests and AI covers everything from today in more depth.
- I'm on LinkedIn, and I'm happy to talk through your setup.
- Now, your questions.